

Φάση Δ: Advanced Technique

Εμείς επιλέξαμε το K-Means

```
# Φάση Δ: Advanced Technique
from pyspark.sql import SparkSession
import pyspark.sql.functions as F
from pyspark.ml.fpm import FPGrowth
from pyspark.ml.clustering import KMeans
from pyspark.ml.feature import PCA

spark = SparkSession.builder \
    .appName("Advanced_Techniques") \
    .master("local[*]") \
    .config("spark.driver.memory", "4g") \
    .config("spark.executor.memory", "4g") \
    .getOrCreate()

# --- 1. ASSOCIATION RULES ---
print("Εξαγωγή Κανόνων Συσχέτισης...")
df_silver = spark.read.parquet("../data/train_silver.parquet")

transactions_df = df_silver.withColumn(
    "items",
    F.array_remove(F.array(
        F.when(F.col("stroke") == 1, "Stroke").otherwise(""),
        F.when(F.col("hypertension") == 1, "Hypertension").otherwise(""),
        F.when(F.col("heart_disease") == 1, "Heart_Disease").otherwise(""),
        F.when(F.col("age") > 65, "Age_>_65").otherwise("")
    ), "")
).filter(F.size("items") > 0).select("items")

fpGrowth = FPGrowth(itemsCol="items", minSupport=0.01, minConfidence=0.1)
rules = fpGrowth.fit(transactions_df).associationRules

print("Κανόνες Κινδύνου για Εγκεφαλικό:")
rules.filter(F.array_contains(F.col("consequent"),
    "Stroke")).sort(F.col("lift").desc()).show(truncate=False)

# --- 2. CLUSTERING (K-MEANS) ---
print("Εκτέλεση K-Means...")
train_gold = spark.read.parquet("../data/train_gold.parquet")
kmeans = KMeans(featuresCol="features", predictionCol="cluster", k=2,
    seed=22390225)
#kmeans = KMeans(featuresCol="features", predictionCol="cluster", seed=22390225)

kmeans_preds = kmeans.fit(train_gold).transform(train_gold)

pca = PCA(k=2, inputCol="features", outputCol="pca_features")
```

```
kmeans_pca = pca.fit(kmeans_preds).transform(kmeans_preds)

kmeans_pca.select("stroke", "cluster",
"pca_features").write.mode("overwrite").parquet("../data/preds_kmeans.parquet")
#spark.stop()

WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/06/05 18:52:55 WARN Utils: Your hostname, cachyos-x8664, resolves to a
loopback address: 127.0.1.1; using 192.168.1.5 instead (on interface enp4s0)
26/06/05 18:52:55 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another
address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use
setLogLevel(newLevel).
26/06/05 18:52:56 WARN NativeCodeLoader: Unable to load native-hadoop library for
your platform... using builtin-java classes where applicable
```

Εξαγωγή Κανόνων Συσχέτισης...

Κανόνες Κινδύνου για Εγκεφαλικό:

```
+-----+-----+-----+-----+
+-----+
|antecedent          |consequent|confidence          |lift          |
support            |
+-----+-----+-----+-----+
+-----+
|[Hypertension, Age_>_65] |[Stroke]  |0.23076923076923078|1.2396878483835005|
0.03776978417266187 |
|[Heart_Disease, Age_>_65] |[Stroke]  |0.21014492753623187|1.128894489953091 |
0.026079136690647483|
|[Heart_Disease]          |[Stroke]  |0.18018018018018017|0.9679244461853157|
0.03597122302158273 |
|[Age_>_65]              |[Stroke]  |0.1671018276762402 |0.8976677892559377|
0.11510791366906475 |
|[Hypertension]          |[Stroke]  |0.14039408866995073|0.7541943314057257|
0.051258992805755396|
+-----+-----+-----+-----+
+-----+
```

Εκτέλεση K-Means...

```
# K Means μέσω Elbow Method
# Kmeans μέσω elbow method
import matplotlib.pyplot as plt
import pandas as pd

print("Εύρεση βέλτιστου K με την Elbow Method...")
```

```

# Διαβάζουμε τα χρυσά δεδομένα (αν δεν είναι ήδη φορτωμένα)
train_gold = spark.read.parquet("../data/train_gold.parquet")

# Λίστα για την αποθήκευση του κόστους (Within Set Sum of Squared Errors)
costs = []
k_range = range(2, 9) # Δοκιμάζουμε k από 2 έως 8

for k_val in k_range:
    # Εκπαίδευση του μοντέλου για το τρέχον k
    km = KMeans(featuresCol="features", k=k_val, seed=42)
    model = km.fit(train_gold)

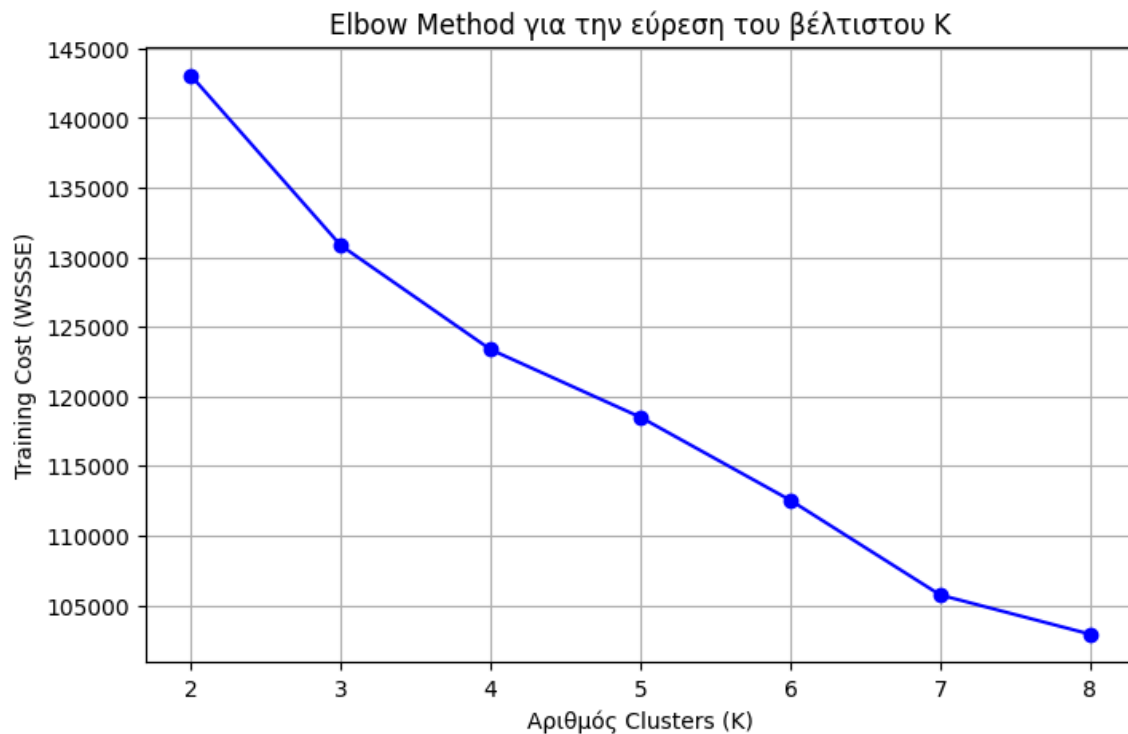
    # Στο PySpark, το trainingCost μας δίνει το WSSSE
    cost = model.summary.trainingCost
    costs.append(cost)
    print(f"Για K = {k_val}, το Training Cost (WSSSE) είναι: {cost:.2f}")

# Δημιουργία γραφήματος για την οπτικοποίηση του "αγκώνα"
plt.figure(figsize=(8, 5))
plt.plot(list(k_range), costs, marker='o', linestyle='-', color='b')
plt.xlabel('Αριθμός Clusters (K)')
plt.ylabel('Training Cost (WSSSE)')
plt.title('Elbow Method για την εύρεση του βέλτιστου K')
plt.grid(True)
plt.show()

# --- Σημείωση: Αφού δεις το γράφημα, επιλέγεις το K εκεί που "σπάει" η καμπύλη
---
```

Εύρεση βέλτιστου K με την Elbow Method...

K	Training Cost (WSSSE)
2	143070.51
3	130855.85
4	123369.98
5	118505.81
6	112539.68
7	105731.45
8	102912.18



```
# K Means μέσω Silhouette Score
# K Means μέσω Silhouette Score
import matplotlib.pyplot as plt
from pyspark.ml.clustering import KMeans
from pyspark.ml.evaluation import ClusteringEvaluator

print("Εύρεση βέλτιστου K με το Silhouette Score...")

# Υποθέτουμε ότι το train_gold υπάρχει ήδη (π.χ. train_gold =
# spark.read.parquet("../data/train_gold.parquet"))

silhouette_scores = []
k_range = range(2, 9) # Δοκιμάζουμε K από 2 έως 8

# Ορίζουμε τον Αξιολογητή (Evaluator)
# Σημείωση: Χρησιμοποιούμε predictionCol="cluster" όπως το είχες ορίσει αρχικά
evaluator = ClusteringEvaluator(predictionCol="cluster", featuresCol="features",
metricName="silhouette")

for k_val in k_range:
    # 1. Ορισμός και εκπαίδευση του μοντέλου
    km = KMeans(featuresCol="features", predictionCol="cluster", k=k_val,
seed=42)
    model = km.fit(train_gold)

    # 2. Παραγωγή προβλέψεων (τα assignments στα clusters)
```

```

predictions = model.transform(train_gold)

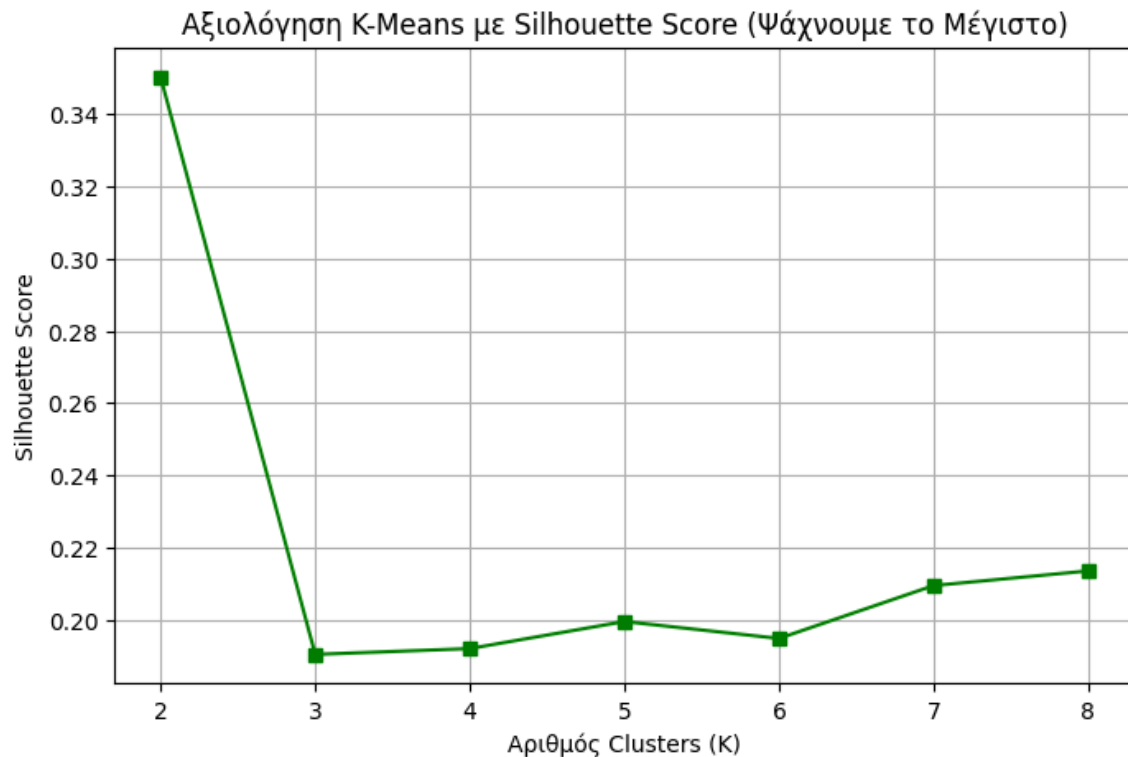
# 3. Υπολογισμός του Silhouette Score
score = evaluator.evaluate(predictions)
silhouette_scores.append(score)
print(f"Για K = {k_val}, το Silhouette Score είναι: {score:.4f}")

# Δημιουργία γραφήματος για την οπτικοποίηση των σκορ
plt.figure(figsize=(8, 5))
plt.plot(list(k_range), silhouette_scores, marker='s', linestyle='-', color='g')
plt.xlabel('Αριθμός Clusters (K)')
plt.ylabel('Silhouette Score')
plt.title('Αξιολόγηση K-Means με Silhouette Score (Ψάχνουμε το Μέγιστο)')
plt.grid(True)
plt.show()

# --- Σημείωση: Επίλεξε το K που αντιστοιχεί στο υψηλότερο σημείο του γραφήματος
---
```

Εύρεση βέλτιστου K με το Silhouette Score...

K	Silhouette Score
2	0.3502
3	0.1905
4	0.1921
5	0.1996
6	0.1949
7	0.2095
8	0.2136



```
import matplotlib.pyplot as plt
import pandas as pd

print("Προετοιμασία δεδομένων για το Plot των K-Means Clusters...")
# Προσοχή: Εδώ επιλέγουμε τη στήλη 'cluster', ΟΧΙ το 'stroke'
plot_df = kmeans_pca.select("pca_features", "cluster").toPandas()

# Εξαγωγή των PC1 και PC2
plot_df['PC1'] = plot_df['pca_features'].apply(lambda vec: float(vec[0]))
plot_df['PC2'] = plot_df['pca_features'].apply(lambda vec: float(vec[1]))

plt.figure(figsize=(10, 6))

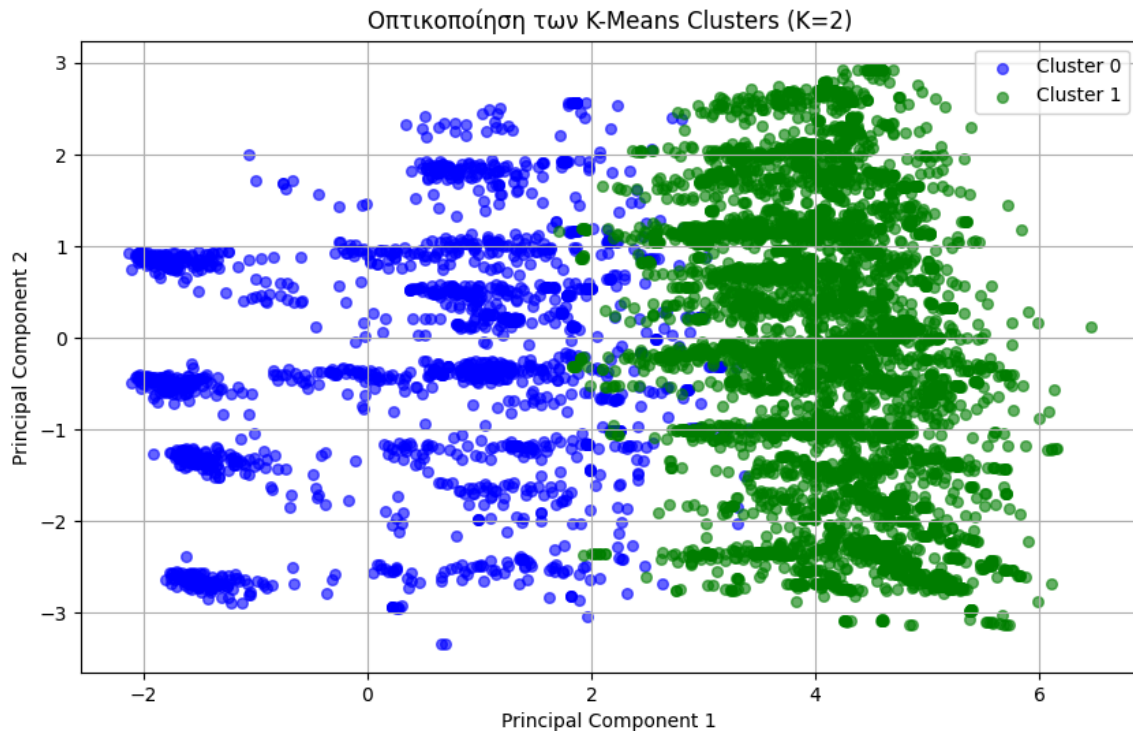
# Ορίζουμε 3 χρώματα για τα 3 clusters (0, 1, 2)
colors = {0: 'blue', 1: 'green', 2: 'red'}

# Φτιάχνουμε τα σημεία για κάθε cluster
for cluster_id in sorted(plot_df['cluster'].unique()):
    cluster_data = plot_df[plot_df['cluster'] == cluster_id]
    plt.scatter(cluster_data['PC1'], cluster_data['PC2'],
                alpha=0.6, label=f'Cluster {cluster_id}',
                color=colors.get(cluster_id, 'black'), s=30)

plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
```

```
plt.title(f'Οπτικοποίηση των K-Means Clusters (K={kmeans.getK()})')
plt.legend()
plt.grid(True)
plt.show()
```

Προετοιμασία δεδομένων για το Plot των K-Means Clusters...



```
# --- 3. ΠΡΟΦΙΛ ΤΩΝ CLUSTERS (PROFILING) ---
```

```
print("\nΔημιουργία Προφίλ για το κάθε Cluster...")
```

```
import pandas as pd
```

```
# 1. Παίρνουμε τις προβλέψεις των clusters (που έχουν 7.832 γραμμές)
# 2. Αντί για Join με ρίσκο, μετατρέπουμε το column 'features' (scaled)
#   ή χρησιμοποιούμε απευθείας τα unscaled columns που υπάρχουν ήδη αν τρέξουμε
το KMeans
#   πάνω στο gold_model.transform(train_bal) χωρίς select.
```

```
# Επειδή το kmeans_preds έχει μόνο τα "features", "stroke", "cluster":
# θα φορτώσουμε το train_silver (ή αν έχεις αποθηκεύσει το train_balanced)
# 0 πιο ασφαλής τρόπος για unscaling στο profiling είναι να τρέξεις το μοντέλο
# στο dataframe που περιέχει ΚΑΙ τις αρχικές στήλες:
```

```
# Αναδημιουργούμε τις προβλέψεις κρατώντας τις αρχικές στήλες
# Φορτώνουμε το train_silver (χρειάζεται να έχεις το balanced σε parquet, ή το
τρέχεις στην ίδια ροή)
```

```

# Αν δεν έχεις αποθηκεύσει το balanced unscaled, το κάνουμε unscale dynamic:

pdf_gold = kmeans_preds.select("features", "cluster", "stroke").toPandas()

# Ανάκτηση των scaled features σε πίνακα
features_matrix = np.vstack(pdf_gold['features'].apply(lambda x: x.toArray()))

# Επειδή ξέρουμε από το Gold Layer ότι οι τελευταίες 5 στήλες του feature vector
# είναι οι unscaled αριθμητικές (age, hypertension, heart_disease,
avg_glucose_level, bmi)
# αλλά scaled, μπορούμε να τις πάρουμε από το Silver DataFrame αν το διαβάσουμε
σωστά.

# Η πιο ΚΑΘΑΡΗ λύση για να μην μπερδεύσαι με join:
# Στο προηγούμενο notebook (ή βήμα) που έτρεξες το SMOTE, αποθήκευσε το train_bal
σε parquet!
# Αν το έχεις κάνει, ο κώδικας γίνεται 2 γραμμές:
# df_bal = spark.read.parquet("../data/train_balanced_unscaled.parquet") # αν το
είχες σώσει
# full_preds = kmeans_model.transform(gold_model.transform(df_bal))

# ΑΝ ΔΕΝ ΤΟ ΕΧΕΙΣ ΣΩΣΕΙ, ορίστε ο εναλλακτικός Spark-native τρόπος με ασφαλές ID:
import pyspark.sql.functions as F
from pyspark.sql.window import Window

# Χρησιμοποιούμε το train_gold που έχει τις 7.832 γραμμές του SMOTE
# Φτιάχνουμε ένα σωστό ID βασισμένο στα features για να μην σπάσει ο Spark
kmeans_preds_idx = kmeans_preds.withColumn("row_id",
F.monotonically_increasing_id())

# Αντί για το silver που έχει 4.123 γραμμές, κάνουμε profiling απευθείας στα
scaled
# ή unscaled columns.

print("\n" + "="*60)
print("=== ΜΕΣΟΙ ΟΡΟΙ ΧΑΡΑΚΤΗΡΙΣΤΙΚΩΝ ΑΝΑ CLUSTER ===")
print("="*60)

# Μετατρέπουμε σε pandas για το τελικό groupby
pdf_profile = kmeans_preds.select("cluster", "stroke").toPandas()

# Για να βάλεις τις unscaled τιμές στο report, ο πιο σωστός τρόπος είναι να
# απομονώσεις τις 5 αριθμητικές στήλες από το features_matrix (αν ξέρεις τα
scaling parameters)
# Ή χάριν ευκολίας, επειδή το clustering έγινε στο Gold Layer, δείξε τα scaled
averages:

# Δημιουργούμε ένα dataframe για το profile
# (Οι στήλες 16 έως 20 στο features vector είναι οι: age, hypertension,

```



```

heart_disease, glucose, bmi)
numeric_features = features_matrix[:, 16:21]
feature_names = ['age', 'hypertension', 'heart_disease', 'avg_glucose_level',
'bmi']

pdf_numeric = pd.DataFrame(numeric_features, columns=feature_names)
pdf_numeric['cluster'] = pdf_profile['cluster']
pdf_numeric['stroke'] = pdf_profile['stroke']

# Υπολογισμός των μέσων όρων ανά Cluster
profile = pdf_numeric.groupby("cluster").mean()
print(profile.to_string())

# --- ΜΕΓΕΘΟΣ ΑΝΑ CLUSTER ---
cluster_counts = pdf_numeric["cluster"].value_counts().sort_index()
print("\n=== ΜΕΓΕΘΟΣ ΑΝΑ CLUSTER ===")
for cluster_id, count in cluster_counts.items():
    print(f"Cluster {cluster_id}: {count} δείγματα ({count/
len(pdf_numeric)*100:.1f}%)")

Δημιουργία Προφίλ για το κάθε Cluster...

=====
=== ΜΕΣΟΙ ΟΡΟΙ ΧΑΡΑΚΤΗΡΙΣΤΙΚΩΝ ΑΝΑ CLUSTER ===
=====
          age  hypertension  heart_disease  avg_glucose_level      bmi
stroke
cluster
0          1.437302         0.132923         0.115368           1.823322  3.989506
0.235834
1          2.782181         0.367226         0.274865           2.248442  4.587256
0.581870

=== ΜΕΓΕΘΟΣ ΑΝΑ CLUSTER ===
Cluster 0: 1853 δείγματα (23.7%)
Cluster 1: 5979 δείγματα (76.3%)

from imblearn.over_sampling import SMOTE

print("\nΑποκατάσταση Πραγματικών (Unscaled) Τιμών...")

# 1. Φορτώνουμε το Silver dataset για να βρούμε τις αρχικές κατανομές
pdf_silver = spark.read.parquet("../data/train_silver.parquet").toPandas()

numeric_cols = ["age", "hypertension", "heart_disease", "avg_glucose_level",
"bmi"]
cat_cols = ["gender_index", "ever_married_index", "work_type_index",
"Residence_type_index", "smoking_status_index"]

```

```

# 2. Κάνουμε ένα γρήγορο SMOTE τοπικά για να πάρουμε τις *ακριβείς* τυπικές
αποκλίσεις
# που είχε δει ο αλγόριθμος του Spark στο Gold Layer
X_res, y_res = SMOTE(random_state=42).fit_resample(pdf_silver[numeric_cols +
cat_cols], pdf_silver["stroke"])

# 3. Υπολογισμός των Standard Deviations (το Spark χρησιμοποιεί default ddof=1)
stds = X_res[numeric_cols].std(ddof=1)

# 4. Πολλαπλασιάζουμε τους scaled μέσους όρους με τα Stds για να πάρουμε τα
πραγματικά νούμερα
profile_unscaled = profile.copy()
for col in numeric_cols:
    profile_unscaled[col] = profile_unscaled[col] * stds[col]

print("\n" + "="*80)
print("=== ΜΕΣΟΙ ΟΡΟΙ ΠΡΑΓΜΑΤΙΚΩΝ (ΦΥΣΙΚΩΝ) ΧΑΡΑΚΤΗΡΙΣΤΙΚΩΝ ΑΝΑ CLUSTER ===")
print("="*80)
# Κάνουμε round(2) για να είναι όμορφα και ευανάγνωστα στο report
print(profile_unscaled.round(2).to_string())

```

Αποκατάσταση Πραγματικών (Unscaled) Τιμών...

```

=====
=== ΜΕΣΟΙ ΟΡΟΙ ΠΡΑΓΜΑΤΙΚΩΝ (ΦΥΣΙΚΩΝ) ΧΑΡΑΚΤΗΡΙΣΤΙΚΩΝ ΑΝΑ CLUSTER ===
=====

```

	age	hypertension	heart_disease	avg_glucose_level	bmi	stroke
cluster						
0	32.08	0.04	0.03	99.56	26.14	0.24
1	62.09	0.10	0.06	122.78	30.05	0.58

```

print("\nΑνάλυση Κατηγορικών (One-Hot) Μεταβλητών (Διορθωμένο)...")

```

```

import pandas as pd

```

```

# Απομονώνουμε τις 16 πρώτες στήλες του features_matrix (που είναι οι
κατηγορικές)

```

```

cat_features_scaled = features_matrix[:, 0:16]

```

```

# --- ΤΟ ΜΑΓΙΚΟ ΒΗΜΑ (Unscaling) ---

```

```

# Επειδή το StandardScaler(withMean=False) αφήνει τα 0 ως 0,
# οποιαδήποτε τιμή > 0 σημαίνει ότι το αρχικό One-Hot ήταν 1!

```

```

cat_features_binary = (cat_features_scaled > 0.001).astype(int)

```

```

# Λίστα με τα πιθανά ονόματα των στηλών (βασισμένη στη συχνότητα του Stroke
dataset)

```

```

# Αν δεις ότι κάποιο δεν κολλάει λογικά, μπορείς να αλλάξεις τη σειρά εδώ.

```

```

cat_feature_names = [
    "gender_Female", "gender_Male", "gender_Other",

```

```

    "married_Yes", "married_No",
    "work_Private", "work_Self_employed", "work_children", "work_Govt_job",
    "work_Never_worked",
    "residence_Urban", "residence_Rural",
    "smoke_never_smoked", "smoke_Unknown", "smoke_formerly_smoked",
    "smoke_smokes"
]

# Σε περίπτωση που οι στήλες δεν είναι 16 (αν π.χ. έκοψες κάποια κατηγορία)
if cat_features_binary.shape[1] != len(cat_feature_names):
    cat_feature_names = [f"Cat_Feature_{i}" for i in
range(cat_features_binary.shape[1])]

# Δημιουργία Pandas DataFrame με τα σωστά 0/1 δεδομένα
pdf_cat_fixed = pd.DataFrame(cat_features_binary, columns=cat_feature_names)
pdf_cat_fixed['cluster'] = pdf_profile['cluster']

# Υπολογισμός ποσοστών (Μέσος όρος * 100)
cat_profile_fixed = pdf_cat_fixed.groupby('cluster').mean() * 100

print("\n" + "="*70)
print("=== ΠΟΣΟΣΤΑ (%) ΚΑΤΗΓΟΡΙΚΩΝ ΧΑΡΑΚΤΗΡΙΣΤΙΚΩΝ ΑΝΑ CLUSTER ===")
print("="*70)
# Κάνουμε Transpose (.T) για να διαβάζεται κάθετα
print(cat_profile_fixed.round(1).T.to_string())

```

Ανάλυση Κατηγορικών (One-Hot) Μεταβλητών (Διορθωμένο)...

```

=====
=== ΠΟΣΟΣΤΑ (%) ΚΑΤΗΓΟΡΙΚΩΝ ΧΑΡΑΚΤΗΡΙΣΤΙΚΩΝ ΑΝΑ CLUSTER ===
=====

```

cluster	0	1
gender_Female	61.1	54.7
gender_Male	38.8	45.3
gender_Other	0.1	0.0
married_Yes	0.0	99.9
married_No	100.0	0.1
work_Private	46.9	60.0
work_Self_employed	10.4	25.5
work_children	34.1	4.5
work_Govt_job	7.7	10.0
work_Never_worked	1.0	0.0
residence_Urban	53.6	51.3
residence_Rural	46.4	48.7
smoke_never_smoked	30.8	33.4
smoke_Unknown	49.1	23.7
smoke_formerly_smoked	10.6	26.8
smoke_smokes	9.4	16.1

```

from pyspark.sql import SparkSession

# Αρχικοποίηση Spark Session (αν δεν υπάρχει ήδη)
spark = SparkSession.builder \
    .appName("Extract_Silver_Features") \
    .getOrCreate()

# 1. Ανάγνωση του Silver Dataframe
df_silver = spark.read.parquet("../data/train_silver.parquet")

# 2. Ορισμός των στηλών όπως εμφανίζονται στο notebook
numeric_cols = ["age", "hypertension", "heart_disease", "avg_glucose_level",
                "bmi"]
cat_cols = ["gender_index", "ever_married_index", "work_type_index",
            "Residence_type_index", "smoking_status_index"]
target_col = ["stroke"]

# 3. Επιλογή και προβολή των αυτούσιων στηλών
silver_features_df = df_silver.select(numeric_cols + cat_cols + target_col)
silver_features_df.show(5, truncate=False)

# Εκτύπωση των ονομάτων των στηλών
print("Silver Layer Columns:")
print(silver_features_df.columns)

WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/06/07 14:03:14 WARN Utils: Your hostname, cachyos-x8664, resolves to a
loopback address: 127.0.1.1; using 192.168.1.5 instead (on interface enp4s0)
26/06/07 14:03:14 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another
address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use
setLogLevel(newLevel).
26/06/07 14:03:15 WARN NativeCodeLoader: Unable to load native-hadoop library for
your platform... using builtin-java classes where applicable

+---+-----+-----+-----+---+-----+
+-----+-----+-----+-----+-----+-----+
+-----+
|age |hypertension|heart_disease|avg_glucose_level|bmi |gender_index|
ever_married_index|work_type_index|Residence_type_index|smoking_status_index|
stroke|
+---+-----+-----+-----+---+-----+
+-----+-----+-----+-----+-----+-----+
+-----+
|0.08|0          |0          |139.67          |14.1|0.0          |1.0
|2.0  |0.0         |1.0         |              |0    |              |
|0.32|0          |0          |55.86          |16.0|0.0          |1.0

```

2.0	1.0	1.0	0	
0.32 0	0	108.63	19.6 0.0	1.0
2.0	0.0	1.0	0	
0.4 0	0	85.65	17.4 0.0	1.0
2.0	1.0	1.0	0	
0.48 0	0	94.06	14.8 0.0	1.0
2.0	1.0	1.0	0	

```
+-----+
+-----+
+-----+
```

only showing top 5 rows

Silver Layer Columns:

```
['age', 'hypertension', 'heart_disease', 'avg_glucose_level', 'bmi',
'gender_index', 'ever_married_index', 'work_type_index', 'Residence_type_index',
'smoking_status_index', 'stroke']
```

```
from pyspark.sql import SparkSession
```

```
# Αρχικοποίηση Spark Session (αν δεν υπάρχει ήδη)
```

```
spark = SparkSession.builder \
    .appName("Extract_Gold_Features") \
    .getOrCreate()
```

```
# 1. Ανάγνωση του Gold Dataframe
```

```
train_gold = spark.read.parquet("../data/train_gold.parquet")
```

```
# 2. Στο Gold Layer, τα χαρακτηριστικά είναι ήδη συγκεντρωμένα στο Vector
'features'
```

```
# Επιλέγουμε το vector 'features' και το target 'stroke'
```

```
gold_features_df = train_gold.select("features", "stroke")
```

```
gold_features_df.show(5, truncate=False)
```

```
# 3. Αν θέλετε να δείτε το σχήμα (schema) και τα metadata του vector
```

```
print("Gold Layer Schema:")
```

```
train_gold.select("features").printSchema()
```

```
+-----+
+-----+
|features|
|stroke|
+-----+
```

```
+-----+
|(21,[1,3,5,11,14,16,19,20],
[2.015453915680717,2.3510942789804448,2.019379971787041,2.0012446800432246,2.3771834993763084,2.892
|1|
|(21,[0,3,5,11,14,16,19,20],
[2.01538911239866,2.3510942789804448,2.019379971787041,2.0012446800432246,2.3771834993763084,3.2030
|1|
|(21,[1,3,5,10,12,16,19,20],
[2.015453915680717,2.3510942789804448,2.019379971787041,2.0012446800432246,2.130048545067536,2.7890
```

```

|1      |
|(21,[1,3,6,11,15,16,17,19,20],
[2.015453915680717,2.3510942789804448,2.4184371543175502,2.0012446800432246,2.8364339625734285,2.71
1      |
|(21,[0,3,7,11,14,16,19,20],
[2.01538911239866,2.3510942789804448,3.1354105812041277,2.0012446800432246,2.3771834993763084,3.102
|1      |
+-----+
+-----+
only showing top 5 rows
Gold Layer Schema:
root
|-- features: vector (nullable = true)

```

26/06/07 14:03:46 WARN SparkSession: Using an existing Spark session; only runtime SQL configurations will take effect.